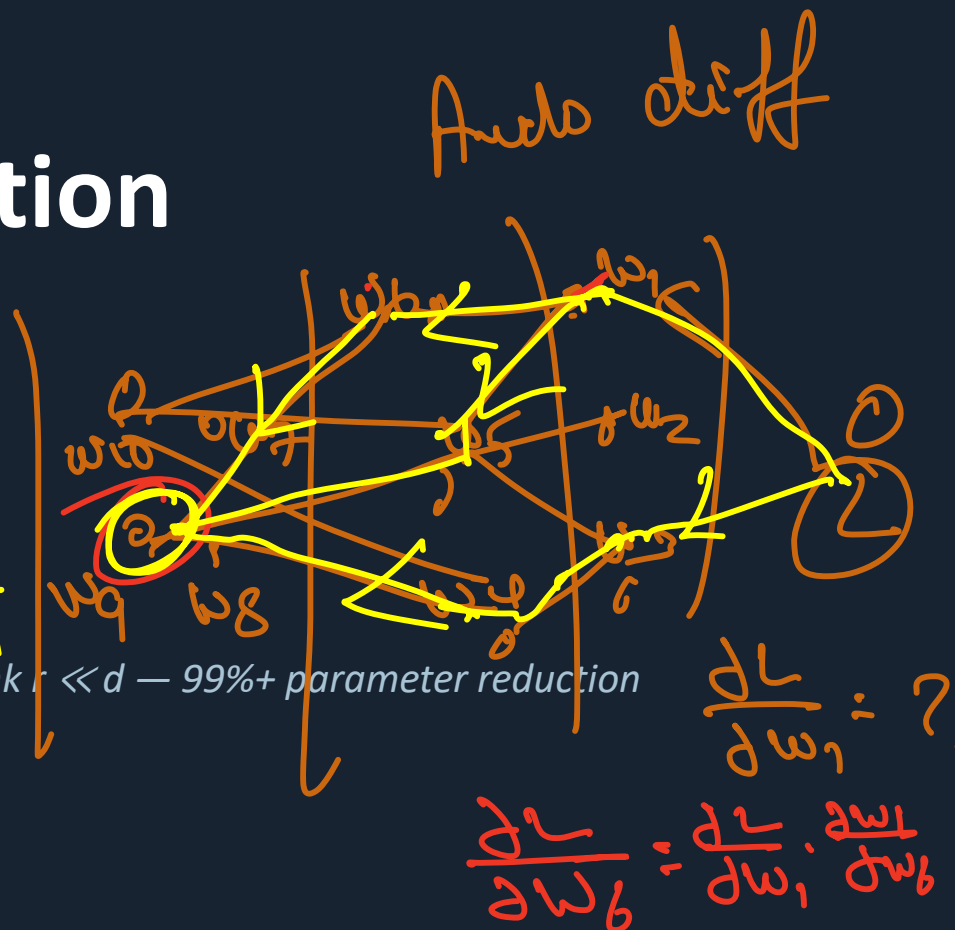


LoRA: Low-Rank Adaptation

$$\frac{\partial L}{\partial w_9} = \frac{\partial L}{\partial w_1} \cdot \frac{\partial w_1}{\partial w_0} \cdot \frac{\partial w_0}{\partial q} + \frac{\partial L}{\partial w_1} \cdot \frac{\partial w_1}{\partial w_5} \cdot \frac{\partial w_5}{\partial w_9} + \frac{\partial L}{\partial w_3} \cdot \frac{\partial w_3}{\partial w_0} \cdot \frac{\partial w_0}{\partial w_9}$$

The dominant PEFT method: freeze W , learn $\Delta W = AB$ where $\text{rank } r \ll d$ — 99%+ parameter reduction



10 min

Duration

B3 – Apply

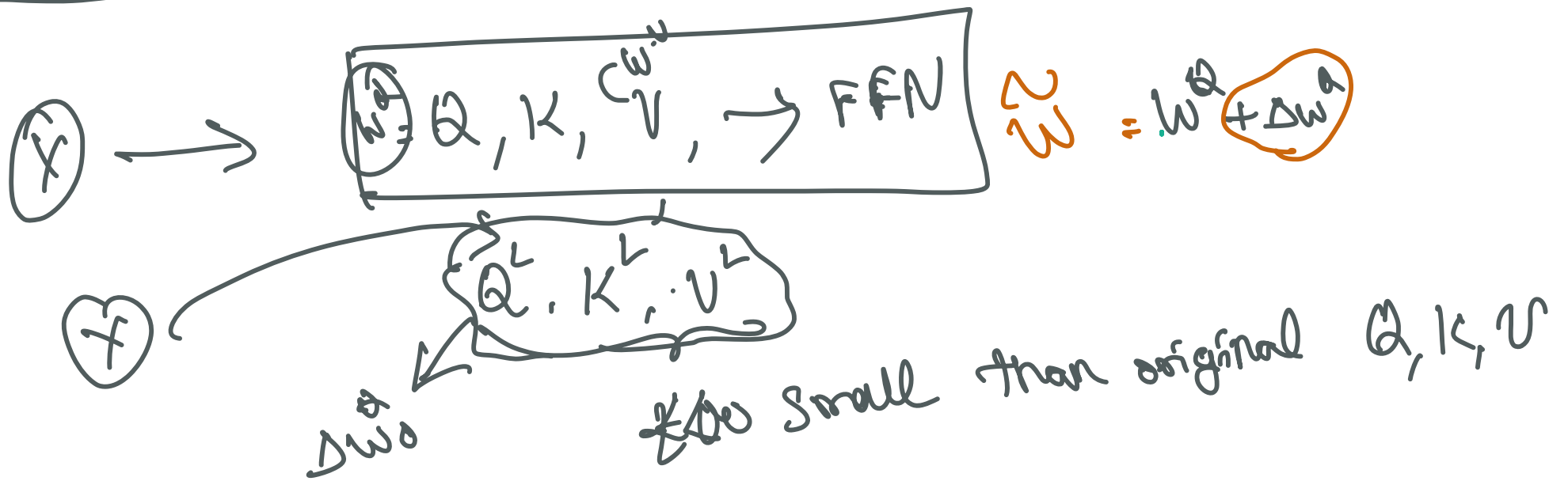
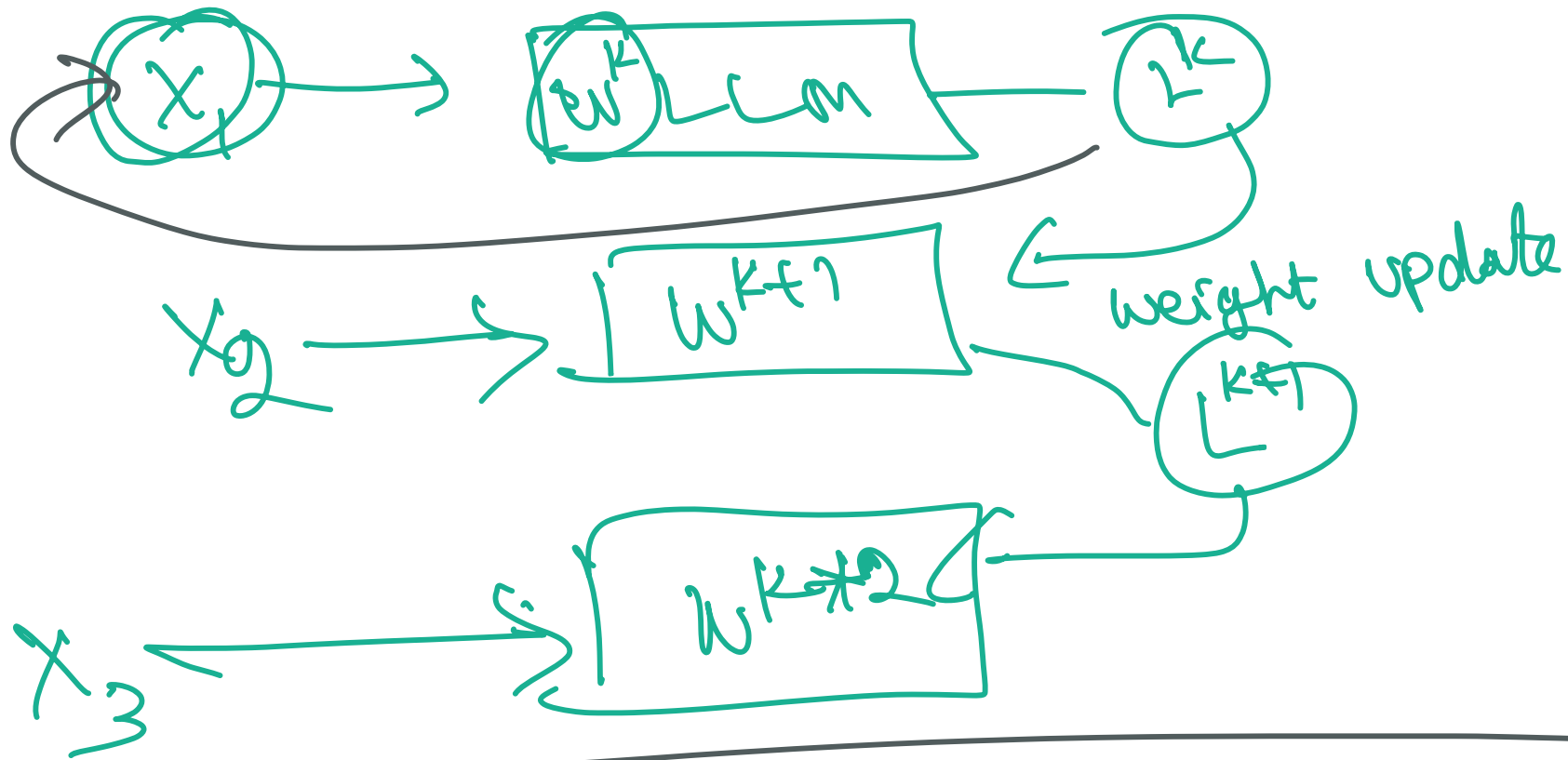
Bloom's

L3 – Advanced

Level

AI-FT-IN-TH-000006

ID



Rank of $A \leq n$.

$$\begin{bmatrix} A \end{bmatrix}_{n \times n} = \begin{bmatrix} B \end{bmatrix}_{n \times n} \begin{bmatrix} C \end{bmatrix}_{n \times n}$$

1024×1024

Regular
 Q, K, V
 in transform

$$\begin{bmatrix} A \end{bmatrix}_{n \times n} = \begin{bmatrix} B' \end{bmatrix}_{n \times p} \begin{bmatrix} C \end{bmatrix}_{p \times n}$$

Suppose $p < n \Rightarrow$ Suppose $p = 1$

LoRA
 Q, K, V
 in transform

Rank = 1

$$\begin{bmatrix} A \end{bmatrix}_{n \times n} = \begin{bmatrix} B \end{bmatrix}_{n \times 1} \begin{bmatrix} C \end{bmatrix}_{1 \times n}$$

1×1024

1024×1

$n = 1024$

2048

LoRA: The Core Mathematical Insight

Key Idea (Hu et al., ICLR 2022)

Instead of updating $W \in \mathbb{R}^{d \times d}$ directly, freeze W and learn a low-rank decomposition: $W' = W + \Delta W = W + AB$, where $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{r \times d}$ with rank $r \ll d$. Only A and B are trained.

- Motivation: weight changes ΔW during fine-tuning have low intrinsic rank — the update lives in a small subspace
- Typical ranks: $r = 4$ to $r = 64$. Most tasks are well-served by $r = 8$ or $r = 16$
- Trainable parameter reduction: for $d=4096$, $r=16 \rightarrow \Delta W$ has $4096 \times 16 + 16 \times 4096 = 131,072$ params vs $4096 \times 4096 = 16.7M$
- LoRA initialisation: A from Gaussian $N(0, \sigma^2)$; B initialised to zero $\rightarrow \Delta W = 0$ at training start (preserves base model)
- After training: merge $W' = W + (\alpha/r) \cdot AB$ directly into $W \rightarrow$ zero inference overhead

LoRA Parameter Count — Llama 3 8B Worked Example

$n \times p$, $p \ll n$.

Model: Llama 3 8B | $d_{\text{model}} = 4096$ | 32 layers | $r = 16$ | targets: W_Q and W_V

Params per LoRA matrix = $d \times r + r \times d = 4096 \times 16 + 16 \times 4096 = 131,072$

Matrices per layer = 2 (W_Q and W_V)

Total LoRA params = $131,072 \times 2 \times 32 \text{ layers} = 8,388,608 \approx 8.4\text{M}$

8.4M LoRA params vs 8,000M base params $\rightarrow 0.105\%$ of total

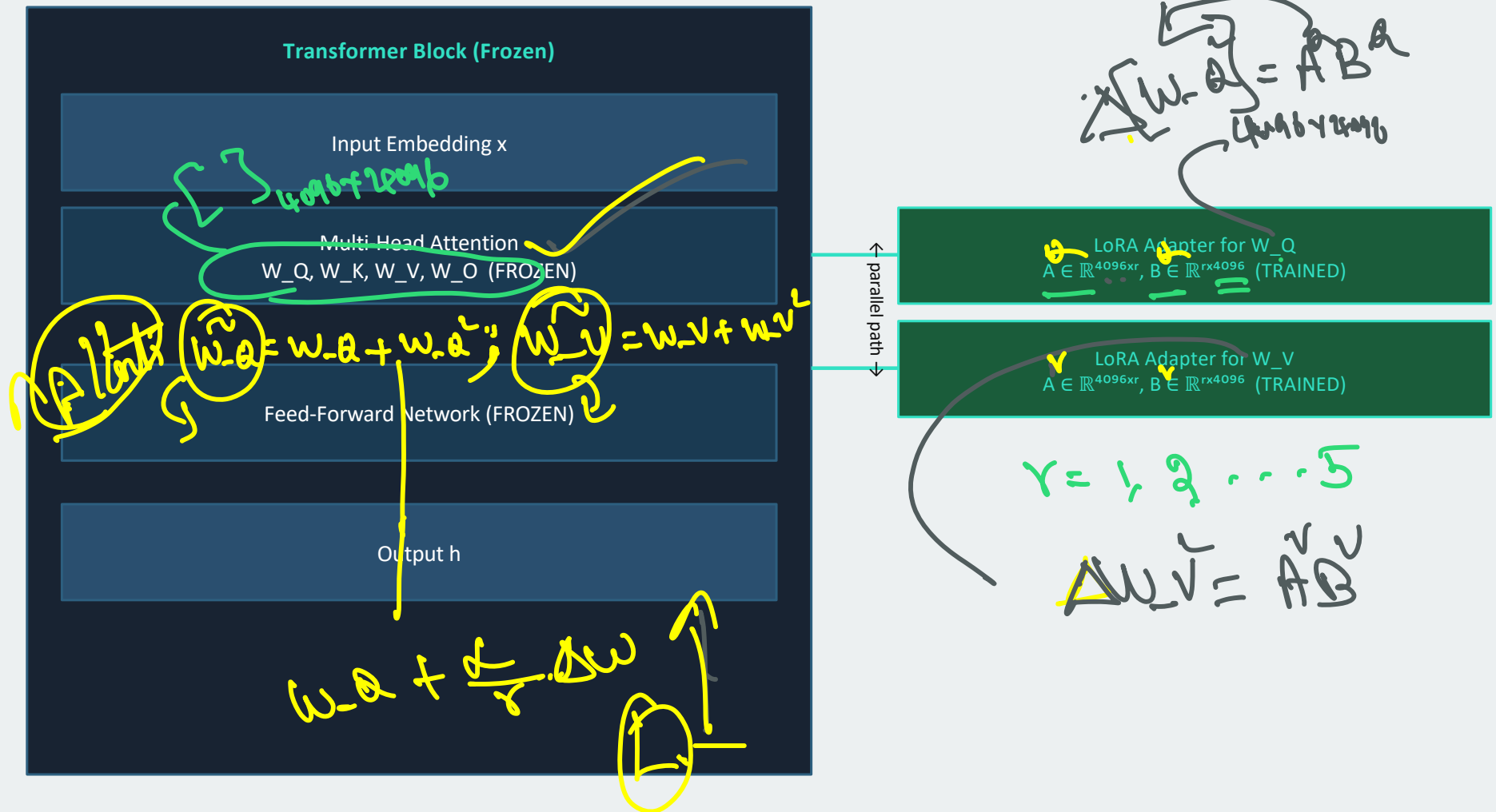
Memory for LoRA training on 8B model:

Base weights (frozen, bf16): $8\text{B} \times 2 \text{ bytes} = 16 \text{ GB}$

LoRA gradients + optim states: $8.4\text{M} \times 12 \text{ bytes} \approx 100 \text{ MB}$

Total $\approx 16.1 \text{ GB} \rightarrow$ Fits on a single RTX 4090 (24 GB)!

LoRA Architecture — Where Adapters Are Inserted



Base model.

Step 1



attach

$$\Delta W^a = \underline{\underline{A \ B}}$$

$n(\log^2)$
 ~~$\neq 0$~~

initialize with it

Step 2
Fine-tune

x^{new}



frozen.

ΔW^a

L

$$\frac{\partial L}{\partial W^a} \rightarrow \Delta W^a = \Delta W - \eta \frac{\partial L}{\partial W^a}$$

Step 3

Option 1:

add ΔW to $W-a$ & deploy $\Rightarrow \tilde{W-a} = W-a + \Delta W$

Option 2:

keep separate $W-a$ & ΔW $\Rightarrow$

LoRA Hyperparameter Guide

Parameter	What It Controls	Common Values	Effect on Quality/Memory
Rank r	Dimensionality of adaptation	4, 8, 16, 32, 64	Higher r = more capacity, more memory
Alpha α	Scaling factor = $\alpha/r \times \Delta W$	16, 32 (often $2 \times r$)	Controls effective learning rate of adapter
Target modules	Which weight matrices get LoRA	q_proj, v_proj (default)	Adding k,o,mlp adds params but helps quality
LoRA dropout	Regularisation on adapter layers	0.0 – 0.1	Helps prevent overfitting on small datasets
Bias training	Train bias terms alongside LoRA	none, lora_only, all	lora_only adds negligible params, mild boost

LoRA Merge — Zero Inference Overhead

```
from peft import PeftModel
from transformers import AutoModelForCausalLM

# Load base model
base_model = AutoModelForCausalLM.from_pretrained("meta-llama/Llama-3-8B")

# Load LoRA adapter on top
lora_model = PeftModel.from_pretrained(base_model, "./lora_adapter/")

# --- Option A: Keep adapter separate (can swap at runtime) ---
outputs = lora_model.generate(input_ids, max_new_tokens=200)

# --- Option B: Merge adapter into base weights (zero overhead) ---
merged_model = lora_model.merge_and_unload()
# merged_model is now a standard Llama 3 8B — no PEFT overhead
merged_model.save_pretrained("./merged_llama3_8b_finetuned/")
```

After merging: $W_{\text{merged}} = W_{\text{base}} + (\alpha/r) \cdot AB$. The adapter is absorbed. Inference is identical in speed to the base model.

KEY REFERENCES

- Hu, E. et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022. arxiv.org/abs/2106.09685
- HuggingFace PEFT: LoRA documentation. huggingface.co/docs/peft/conceptual_guides/lora
- Aghajanyan, A. et al. (2020). Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning. arXiv.

 **Next: Concept 8: QLoRA — Quantised LoRA (AI-FT-IN-TH-000007)**